Implementação - Relacionamento

Orientação a Objetos – DCC025

2012.3

Relacionamento Entre Classes

- Associação
  - A classe dependente possui um atributo que é uma referência para a outra classe. Também chamada de dependência por atributo (ou estrutural)

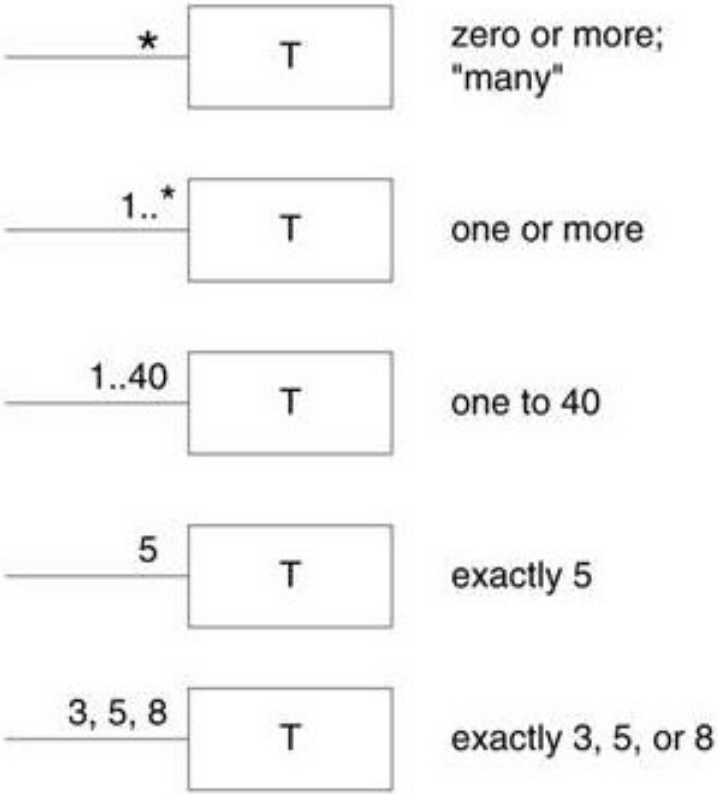
- Dependência
  - Indica que uma classe depende dos serviços (operações) fornecidos por outra classe.

Implementação - Associação

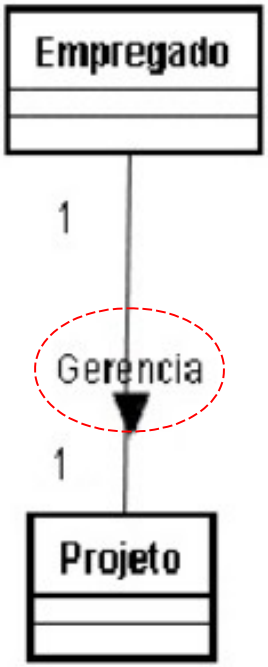
Associação

- Associações
  - Bidirecional
  - Unidirecional

Associação


*
T
zero or more;
"many"
1..*
T
one or more
1..40
T
one to 40
5
T
exactly 5
3, 5, 8
T
exactly 3, 5, or 8

Associação

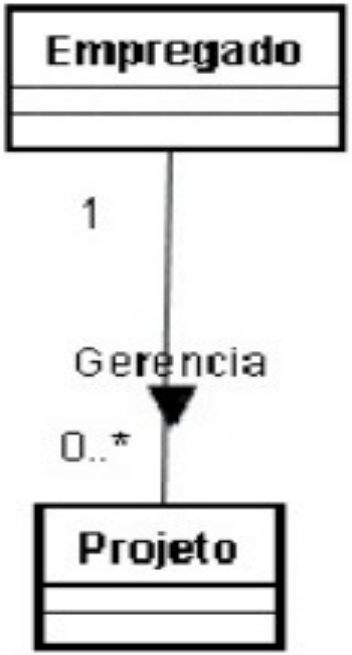



```
public class Empregado {

    private Projeto projeto;
}
```

Sentido de Leitura

```
public class Projeto {

    private Empregado empregado;
}
```

Associação



Exemplo de associação

Observe que um Empregado pode gerenciar nenhum ou vários projetos. Estamos lidando aqui com o conceito de coleções

```
import java.util.ArrayList;

public class Empregado {

    private ArrayList<Projeto> projetos = new ArrayList<Projeto>();
}
```

Abre parênteses -> Coleções

Coleções

- Estruturas de dados pre-definidas

- Oferecem metodos eficientes que:
  - Organizam,
  - Armazenam e,
  - Recuperam os dados.

  Ex: ArrayList (Classe genérica)

ArrayList

Métodos comuns:

- **add** - adiciona elemento ao fim do ArrayList
- **get** – retorna o elemento no indice especificado
- **contains** – retorna true se ArrayList contiver o elemento especificado
- **clear** – remove todos os elementos do ArrayList
- **remove** – remove o elemento no indice especificado
- **size** – retorna o numero de elementos armazenados no ArrayList

ArrayList

Exemplo Bloco de Notas

Contrua uma classe capaz de armazenar anotações, exibir e remover uma determinada anotação.

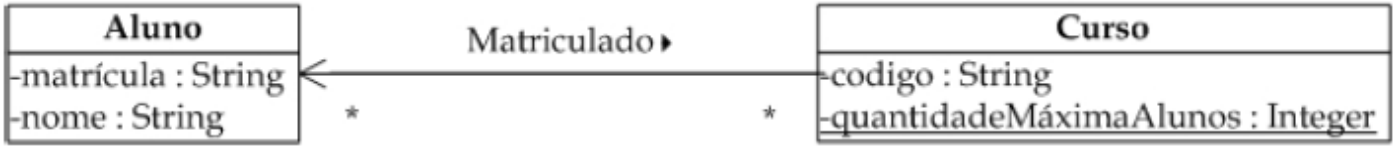
Coleções <- Fecha parênteses

Associações

- Associações
  - Associações podem ser bidirecionais ou unidirecionais.
  - Bidirecional
    - Indica que há um conhecimento mútuo entre os objetos associados.
  - Unidirecional
    - Indica que apenas um dos extremos da associação tem ciência da existência da mesma. Representada através da adição de um sentido à seta da associação.

Associações

- Associações
  - A escolha da navegabilidade de uma associação pode ser feita através do estudo dos diagramas de interação. O sentido de envio das mensagens entre objetos influencia na necessidade ou não de navegabilidade em cada um dos sentidos.

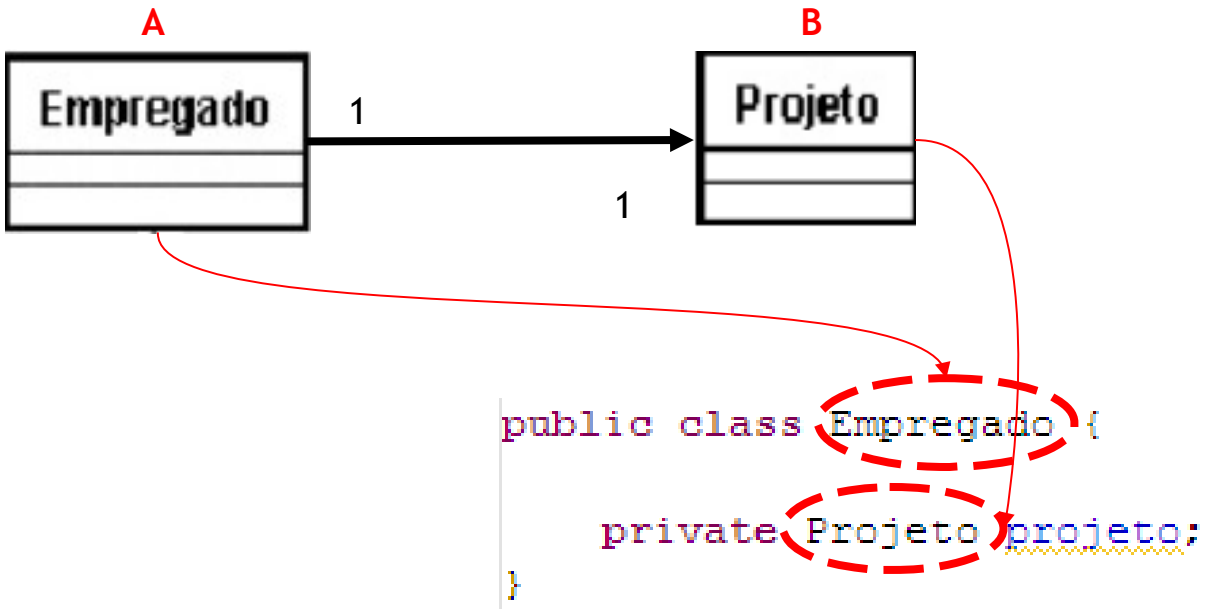


Implementação de Associações

- Há três casos
  - Em função da conectividade: 1:1, 1:N e N:M
  - Para uma associação 1:X entre duas classes A e B:
    - Se a navegabilidade é unidirecional no sentido de A para B, é definido um atributo do tipo B na classe A.Se a navegabilidade é bidirecional, podemos aplicar o procedimento acima para as duas classes.

Relacionamento Unidirecional

Unidirecional Para 1



Unidirecional Para 1

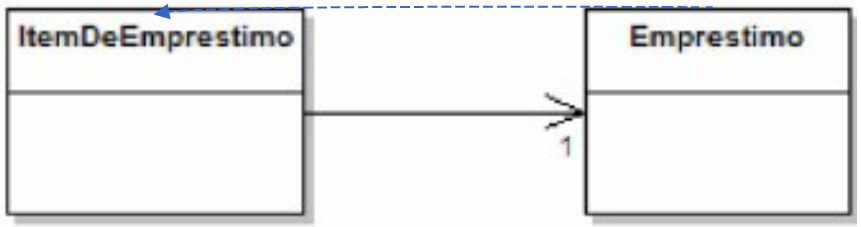
Unidirecional

Como a associação 1 é obrigatória para o objeto origem,construtor da classe deve ter como parâmetro o elemento a ser associado, para que, desde o momento da criação, todas as instâncias da classe na origem da associação estejam consistentes.

Unidirecional Para 1

Construtor de ItemDeEmprestimo

```
class itemDeEmprestimo
{
        private Emprestimo emprestimo;

        public ItemDeEmprestimo(Emprestimo emprestimo)
        {
                this.associaEmprestimo(emprestimo)
        }

        public void associaEmprestimo(Emprestimo emprestimo)
        {
                This.emprestimo = emprestimo;
        }

        public Emprestimo getEmprestimo()
        {
                return emprestimo;
        }
}
```

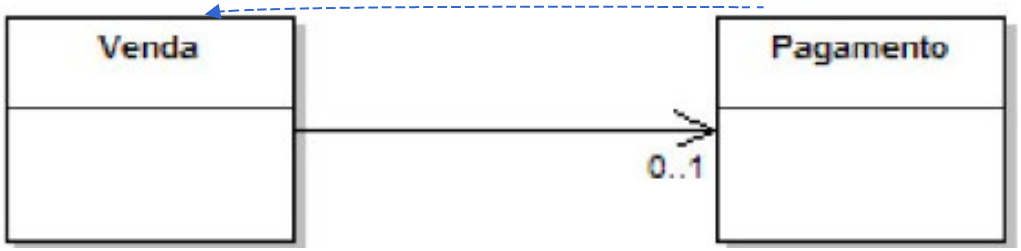


Unidirecional Para 0:1

Não é necessário passar um objeto como parâmetro para o método construtor, pois a associação 0..1 não é obrigatória

Unidirecional Para 0:1

```
class Venda
{
    private Pagamento pagamento;

    public Venda() {}

    public void associaPagamento(Pagamento pagamento)
    {
            this.pagamento = pagamento;
    }

    public void desassociaPagamento()
    {
            This.pagamento = null;
    }

    public Pagamento getPagamento()
    {
            return pagamento;
    }
}
```

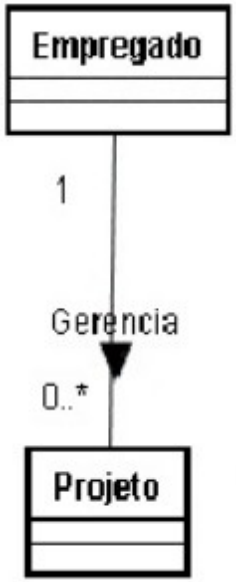


Unidirecional Para 0:1

```
public class Professor {
  private GradeDisciplinas grade;
  ...
}
```

Unidirecional Para *

- Para uma associação 1:N entre duas classes A e B:
  - São utilizados atributos cujos tipos representam coleções de elementos.
  - 1:N: essas coleções são codificadas na classe de multiplicidade 1



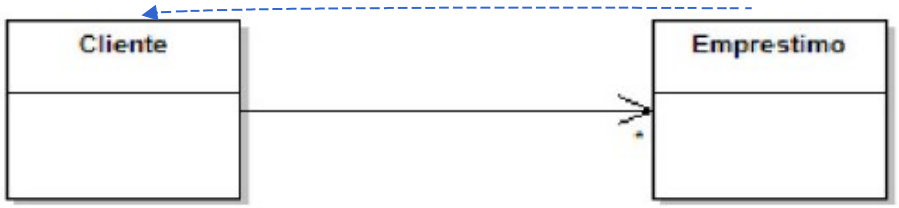

```
import java.util.Set;

public class Empregado {

    private Set<Projeto> projetos;
}
```

Unidirecional Para *

A implementação de uma associação desse tipo constitui-se pela implementação de uma variável do tipo Conjunto, no código Java da classe Cliente.

Unidirecional Para *

```
class Cliente {
        private Set emprestimos = new HashSet();
        public Cliente () { }
        public void adicionaEmprestimo(Emprestimo emprestimo)
        {
                this.emprestimos.add(emprestimo);
        }
        public void removeEmprestimo(Emprestimo emprestimo)
        {
                this.emprestimos.remove(emprestimo);
        }
        public Set getEmprestimos ()
         {
                return Collections.unmodifiableSet(emprestimos);
        }
}
```



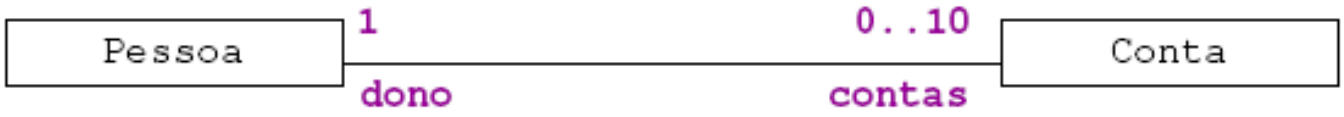
Unidirecional Para *

```
public class Aluno {
  private Set<Participacao> participacoes;
  ...
```

Unidirecional Para *

```
public class Aluno {
    private Set<Participacao> participacoes;
    ...

    public boolean adicionarParticipacao(Participacao p){
        ...
    }

    public boolean removerParticipacao(Participacao p) {
        ...
    }
}
```

Unidirecional – Valores Específicos




```
public class Pessoa {
    int numProxConta=0;
    static final int numMaxConta=10;
    Conta[] conta;
    ...
    Pessoa() {
        conta = new Conta[numMaxConta];
    }
    void associarConta(Conta conta){
        if (numProxConta<numMaxConta)
            this.conta[numProxConta++] = conta;
        else
            System.out.println("Número máximo atingido!");
    }
}
```

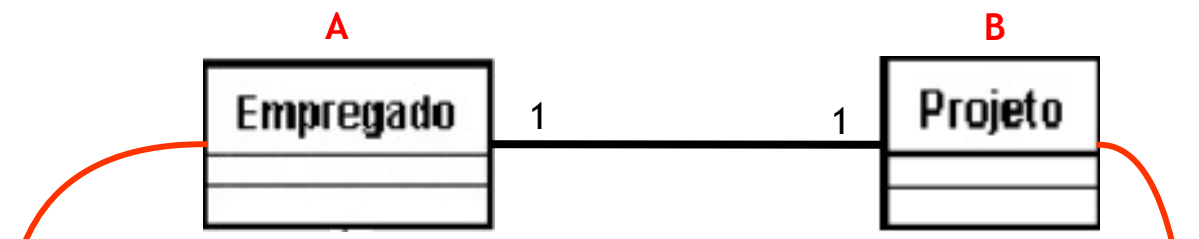
```
public class Conta{
    Pessoa dono;
    ...
    Conta(Pessoa dono){
        this.dono = dono;
    }
}
```

Relacionamento Bidirecional

Associação Bidirecional

- Associação bidirecional
  - Deve ser implementada em ambas as classes.
  - Independente de a multiplicidade ser para 1, para 0..1 ou para *

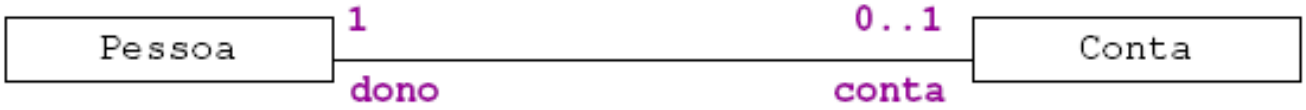
Associação Bidirecional 1:1




```
public class Empregado {

    private Projeto projeto;
}
```

```
public class Projeto {

    private Empregado empregado;
}
```

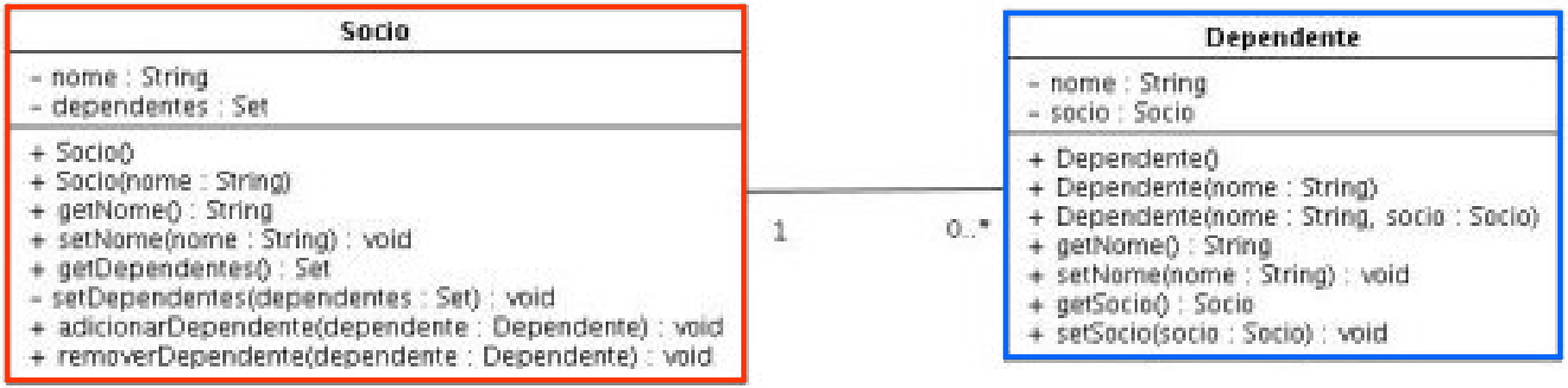
Bidirecional Para 0:1




```
public class Pessoa {
   Conta conta;
   ...
   Pessoa(){
        conta = null;
         ...
   }
   void associarConta(Conta conta) {
        this.conta = conta;
   }
}
```

```
public class Conta{
   Pessoa dono;
   ...
   Conta(Pessoa dono){
        this.dono = dono;
         ...
   }
}
```

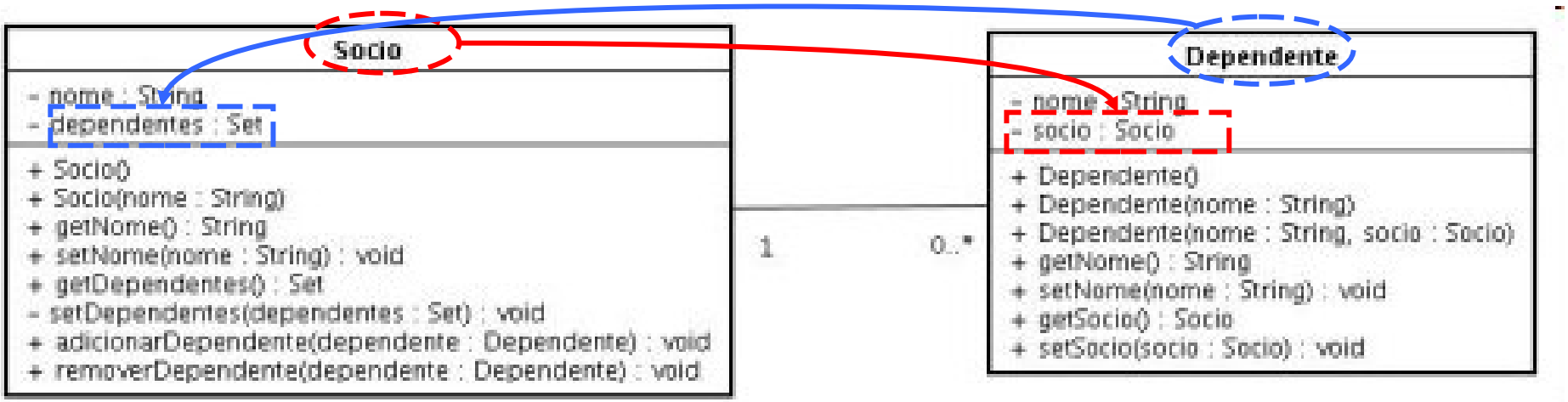
Associação Bidirecional 1:N




```
public class Socio {
        private String nome;
        private Set dependentes;
```

```
public class Dependente {
        private String nome;
        private Socio socio;
```

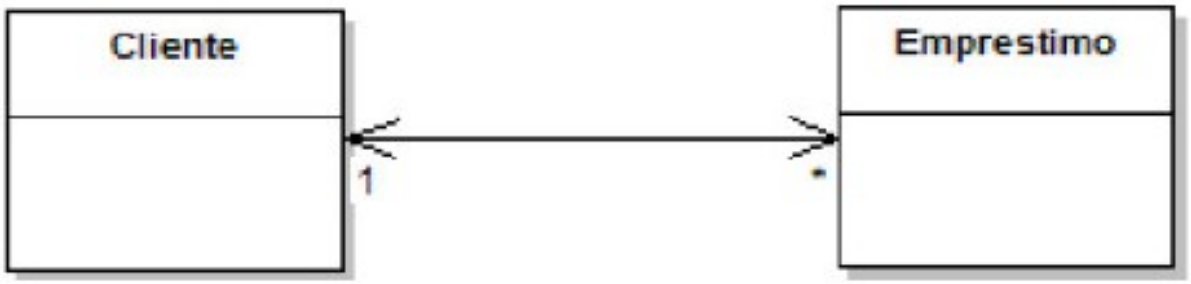
Implementação de Associações




```
public class Socio {
        private String nome;
        private Set dependentes;
```

```
public class Dependente {
        private String nome;
        private Socio socio;
```

Associação Bidirecional 1:N

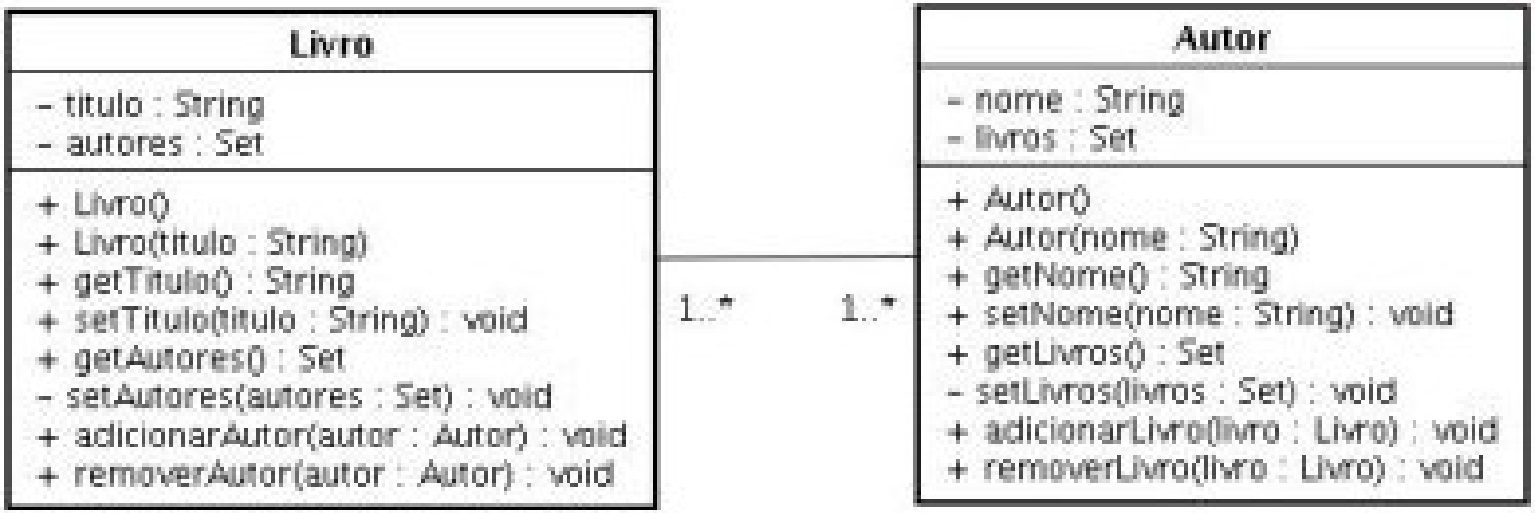


Associação Bidirecional 1:N

```
class Cliente
{
        private Set emprestimos = new HashSet();

        public Cliente () {}
```

```
class Emprestimo
{
        private Cliente cliente;

        public Emprestimo(Cliente cliente)
        {
                this.associaCliente(cliente);
        }
```

Associação Bidirecional N:N



- Implementar N:M
  - Insira, em ambas as classes, um atributo do tipo coleção
  - Insira os métodos get e set correspondentes ao atributo coleção;
  - Insira métodos para adicionar e remover objetos na coleção

Implementação de Associações

```
public class Livro {

private String titulo;
private Set autores;

...
}
```

1

```
public void setAutores(Set autores) {
...
}

public Set getAutores() {
...
}
```

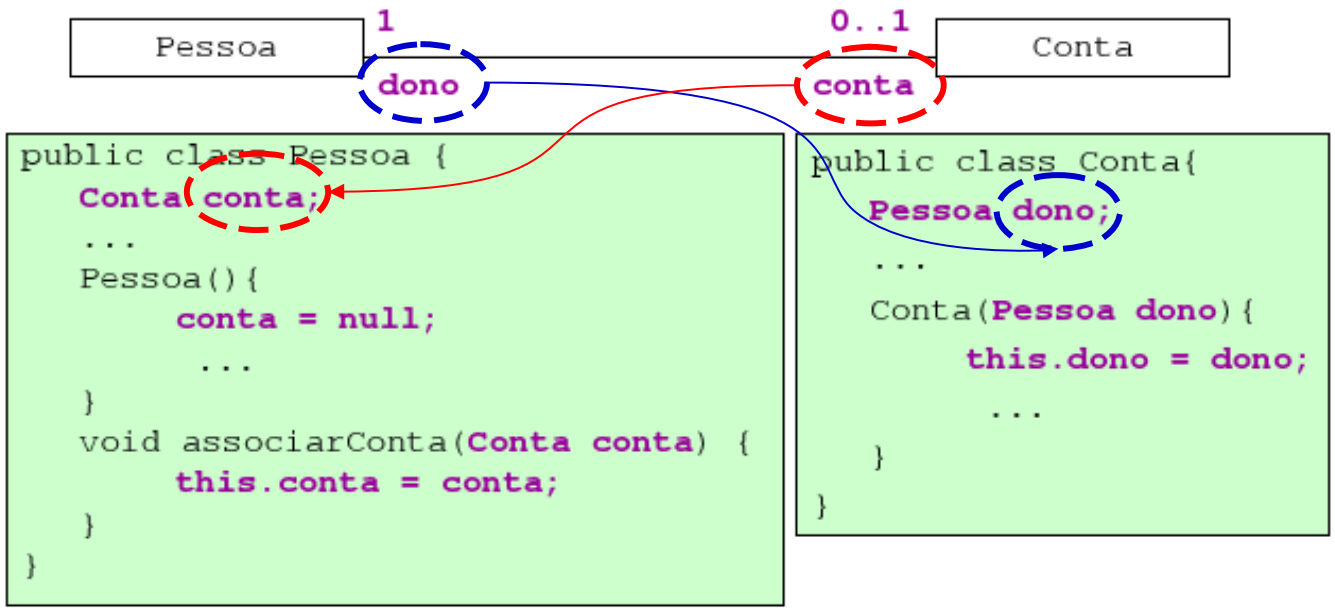
2

```
public void adicionarAutor(Autor autor){
...
}

public void removerAutor(Autor autor){
...
}
```

3

Associação - Continuação

Papéis

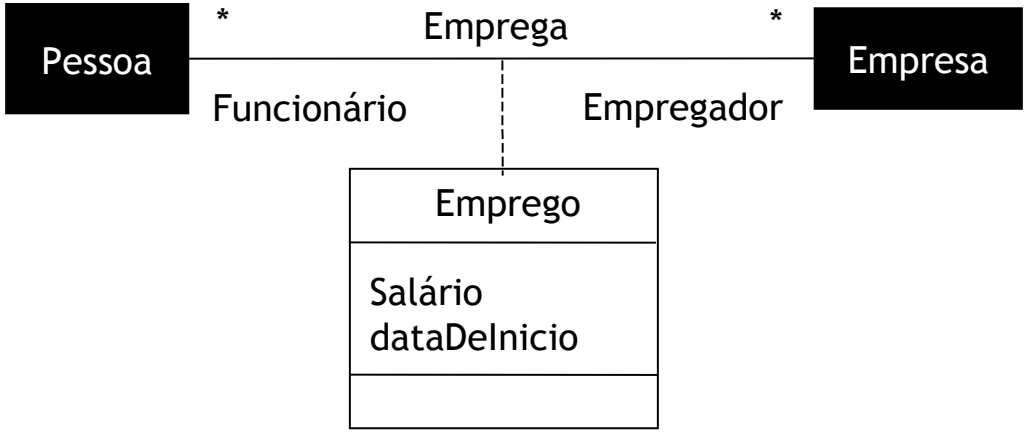

Pessoa
1
dono
0..1
conta
Conta
public class Pessoa {
Conta conta;
...
Pessoa(){
conta = null;
...
}
void associarConta(Conta conta) {
this.conta = conta;
}
}
public class Conta{
Pessoa dono;
...
Conta(Pessoa dono){
this.dono = dono;
...
}
}

Papéis

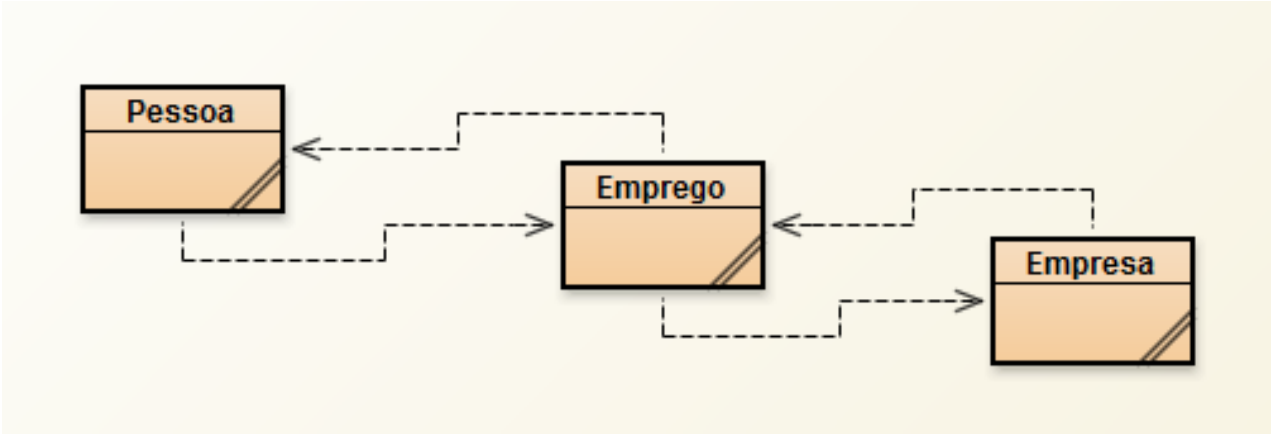
```
public class Pessoa {
   Conta conta;
   Endereço reside;
   ...
   void Pessoa(Endereço endereço) {
        conta = null;
        reside = endereço;
         ...
   }
}
```

```
public class Endereço {
   String rua;
   ...
}
```

Classe Associativa



Classe Associativa



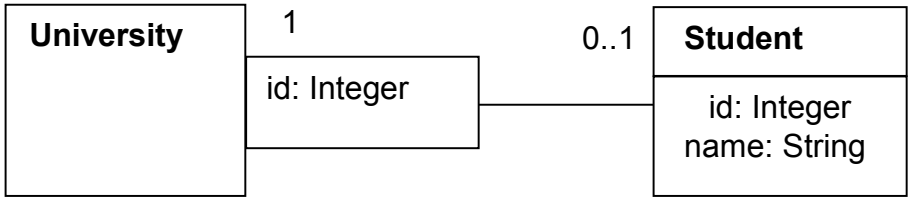
Classe Associativa

```java
import java.util.ArrayList;

public class Empresa
{
    private ArrayList<Emprego> empregos;

    public Empresa(){
        empregos = new ArrayList<Emprego>();
    }
}
```

```java
import java.util.ArrayList;

public class Pessoa
{
    private ArrayList<Emprego> empregos;

    public Pessoa(){
        empregos = new ArrayList<Emprego>();
    }
}
```

```java
import java.util.Date;

public class Emprego
{
    private Empresa empregador;
    private Pessoa funcionario;
    private double Salario;
    private Date dataInicio;

}
```

Associação Qualificada

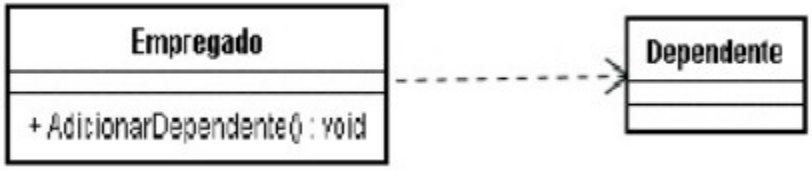



```java
import java.util.Hashtable;

public class University
{
    private Hashtable students;

    public void addStudent(Student s){
        students.put(s.getId(), s);
    }

    public void removeStudent(int id){
        students.remove(id);
    }

    public Student getStudent(int id){
        return (Student)students.get(id);
    }
}
```

```java
public class Student
{
    private int id;
    private String nome;

    public int getId(){
        return id;
    }
}
```

Implementação - Dependência

Dependência

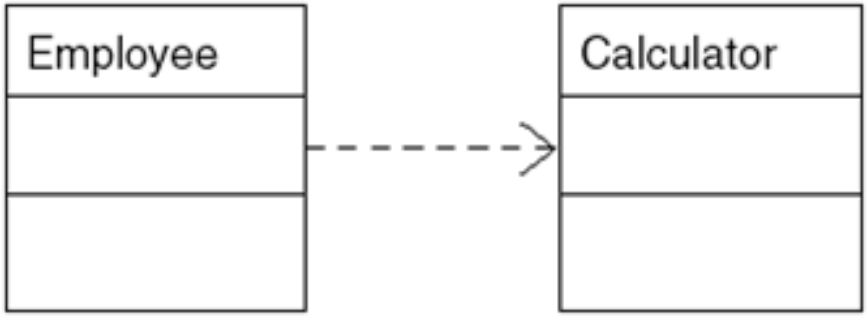



```
public class Empregado {

    public void adicionarDependente(Dependente dependente){
        //implementação do método
    }
}
```

Dependência por atributo

Dependência

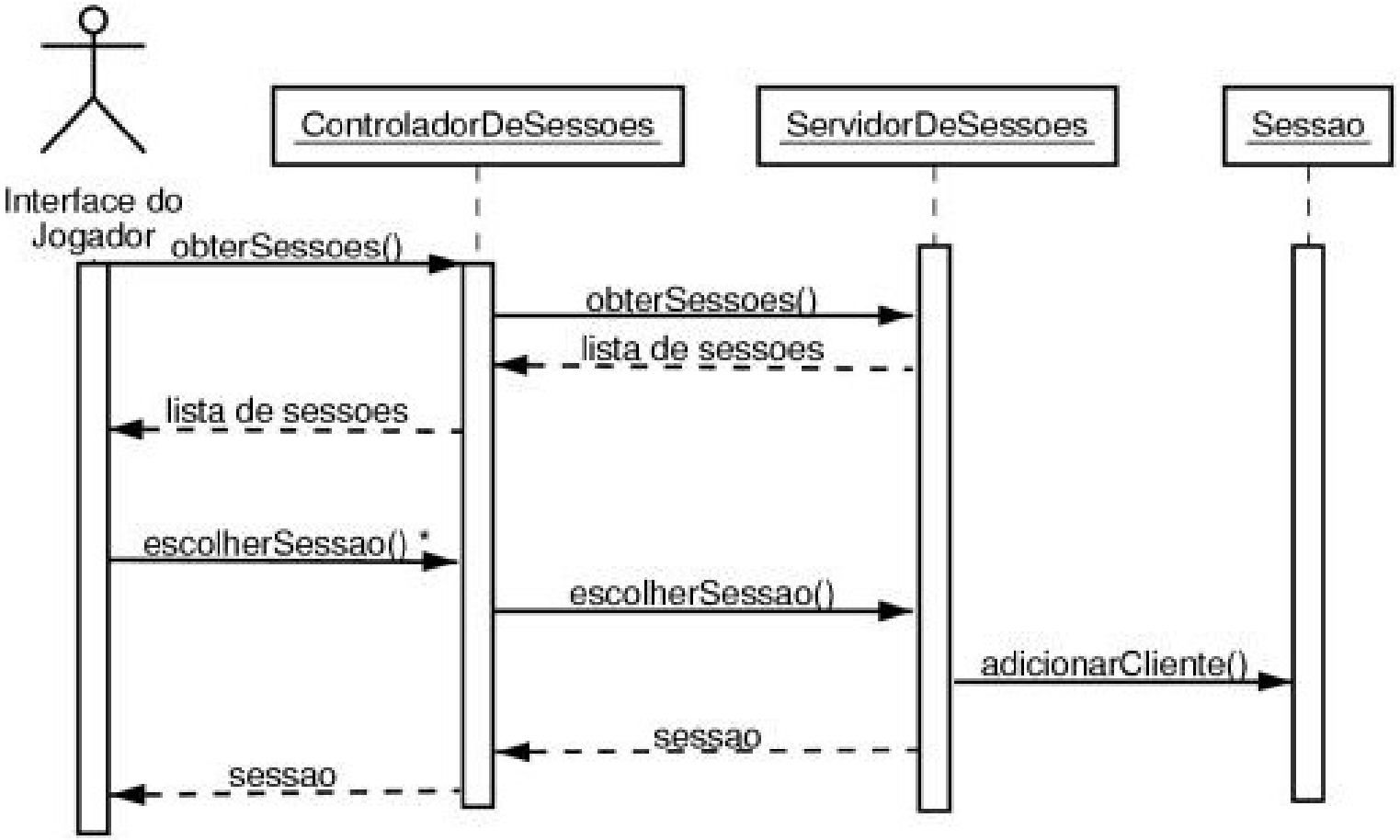
```
public class Employee {
        public void calcSalary(Calculator c) {
        ...
        }
}
```



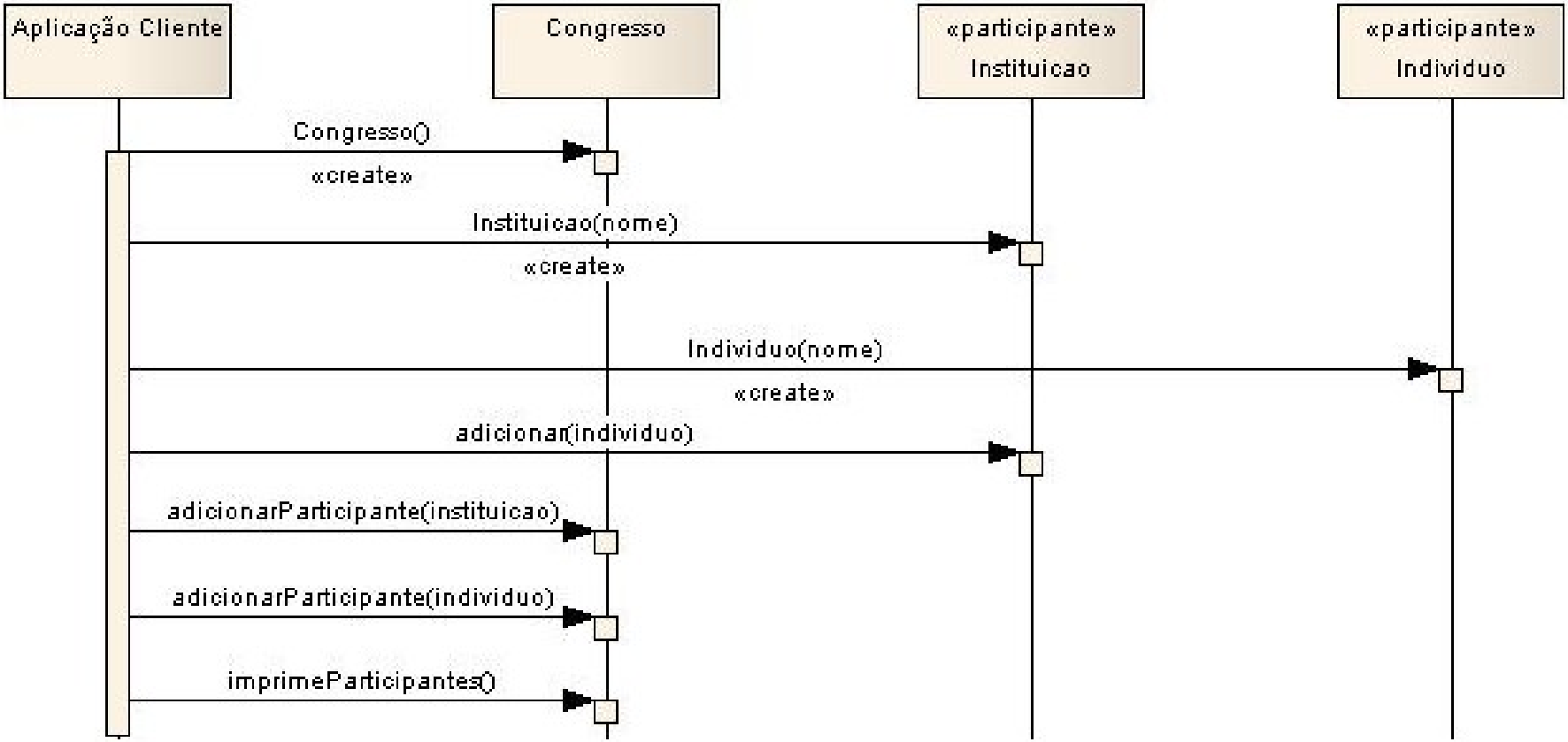
Encontrar Relacionamentos

Relações UML

Diagrama de Sequência



Relações UML



Implementação - Relacionamento

Orientação a Objetos – DCC025

2012.1